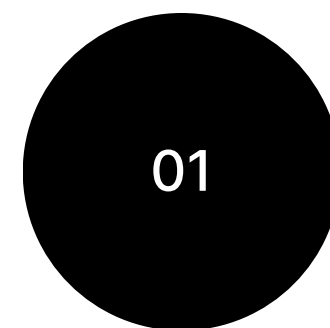


Sistema IoT Distribuido con FreeRTOS: Sensado, Control y Telemetría vía MQTT

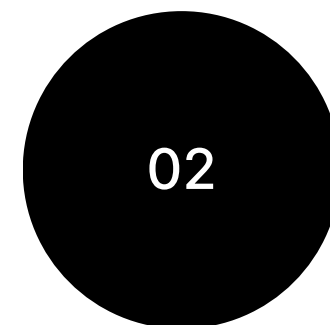


Manuel Fernández Giráldez
Nuria Sorrentino Rebull
Daniel González Domínguez

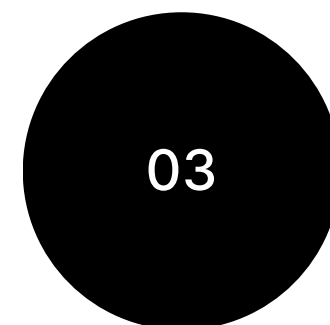
ÍNDICE



REQUISITOS DE
DISEÑO



PRESTACIONES Y
DECISIONES DE
DISEÑO



ARQUITECTURA DE
COMUNICACIONES

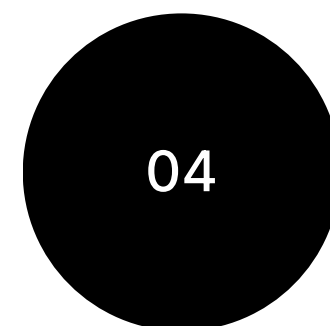
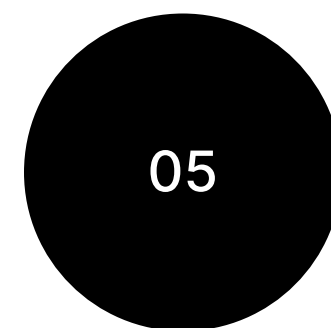
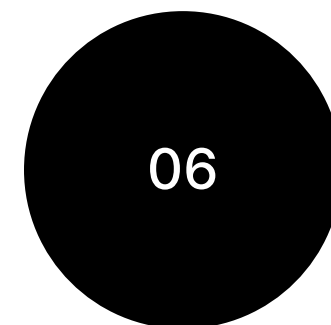


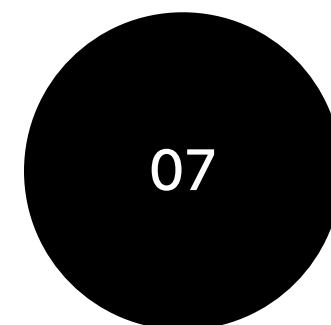
DIAGRAMA DE
BLOQUES



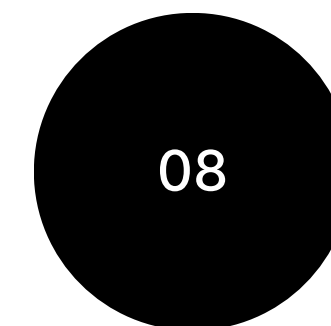
TAREAS
COMUNES



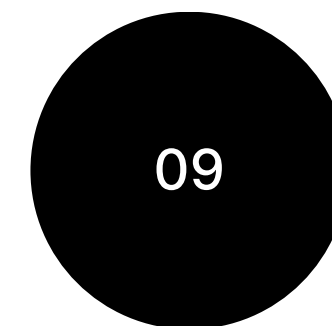
NODO 1: ACCEL



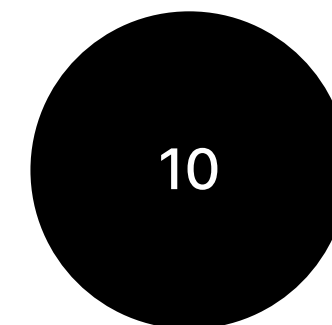
NODO 2: ENV



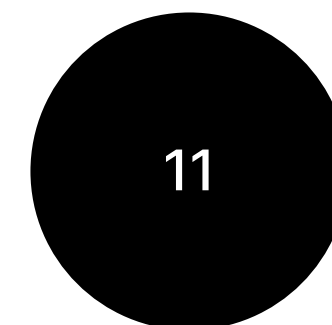
INTERFAZ DE
CONTROL



LOGS POR MQTT



CONCLUSIONES



MEJORAS
FUTURAS

REQUISITOS DE DISEÑO

02

REQUISITOS OBLIGATORIOS

- Implementación de dos nodos:
 - Nodo 1 Acelerómetro
 - Nodo 2 Medidor de entorno
- Uso de sistema operativo en tiempo real: FreeRTOS
- Minimizar y reutilizar las tareas y colas
- Uso de notificaciones:
 - Aviso de nuevas configuraciones
- Uso de RTC configurado para alarmas/mediciones periódicas
- Puerto de depuración y puerto de configuración por UART

REQUISITOS OPCIONALES

- Configuración de RTC por UART/MQTT
- Gestión dinámico del WiFi, no requiere recompilación
- Doble canal de comunicación y configuración:
 - Topic de MQTT
 - UART
- Logging dual:
 - Topic específico para logs
 - Logs por puerto serie

PRESTACIONES Y DECISIONES DE DISEÑO

03

01

COMUNICACIÓN MINIMALISTA: DOS COLAS

- **qMqttTx** (8 x 1060 Bytes): se llena con los mensajes de todos los publicadores. Se parametriza de forma que identifica Topic objetivo y payload
- **qCmdRx**: recepción de los comandos de control UART/MQTT

02

ESTRUCTURA DE DATOS PARAMETRIZABLE

- Se usa una estructura con dos campos:

```
typedef struct {  
    char topic[32];        // Destino dinámico  
    char payload[1024];    // Datos en JSON  
} MqttMsg_t;
```

- Multipropósito: sirve para Accel, Env, Logs, Comandos...
- **Desacoplamiento**: las tareas NO conocen protocolos, solo encolan datos

03

USO DE JSON TOBUSTO

- Formato autodescriptivo: {"t0":123,"fs":52,"s":0,"n":16,"z":[...]}
- Creación de **Chunks** de tamaño 64 muestras en el modo continuo. Cuando ocurre, se reconstruye con el campo **s** (secuencia) y **n** (total) para reensamblar

04

PARSER DE COMANDOS MODULAR

- Único punto de entrada:
 - **Cmd_ParseLine()** para UART y MQTT
 - Extensible, se pueden añadir nuevos comandos sin modificar la infraestructura ni interfaces

ARQUITECTURA DE COMUNICACIONES

04



Comandos (control)

- `bridge.lcmd/1` → comandos para Nodo 1 (ACC)
Ejemplos payload, CONT ON, CONT OFF, READ, WIFI <ssid> <Pass>, RTC ..



Datos (publicación)

- `bridge/accel/1` → datos acelerometro del Nodo 1 (JSON con bloques de muestras)
- `bridge/env/2` → datos entorno del Nodo 2 (JSON temp/hum, id, mg, motivo, etc.)



Logs (debug, opcional)

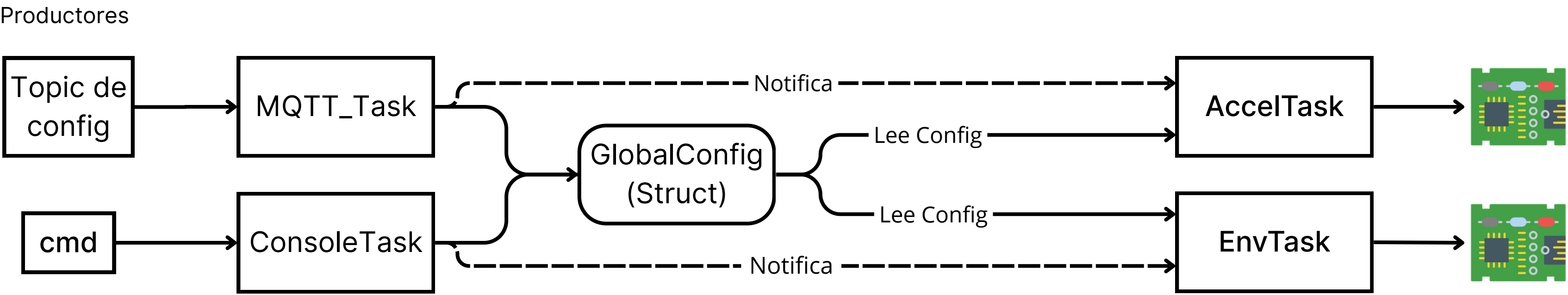
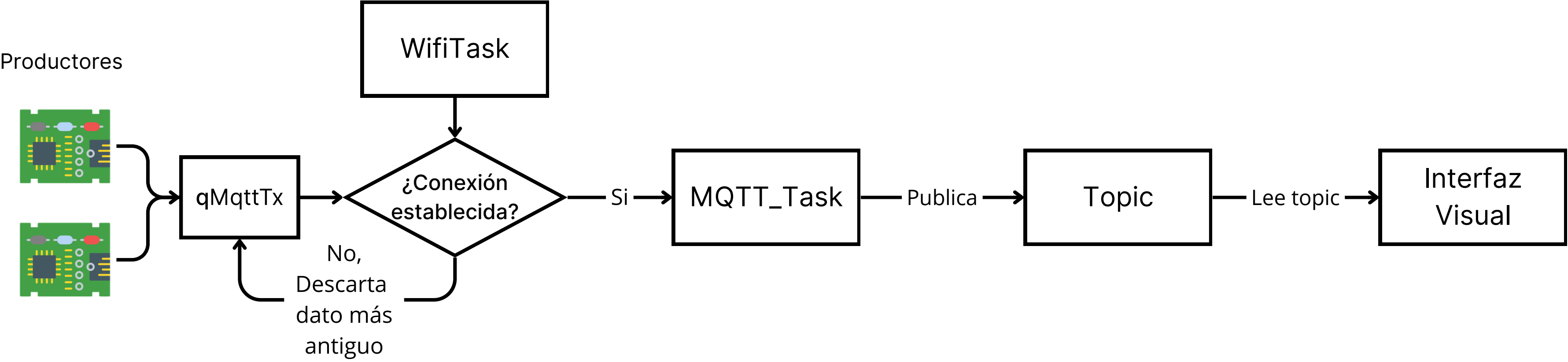
- `bridge/log/1` → logs del Nodo 1
- `bridge/log/2` → logs del Nodo 2



Comunicación clave ENV → ACC

- ENV publica en `bridge/cmd/1` el payload CONT ON/OFF para activar/desactivar el modo continuo del acelerometro.

DIAGRAMA DE BLOQUES



TAREAS COMUNES: WiFi, MQTT y Configuración

06

WIFI TASK

Stack: 2048 Bytes
Prioridad: Normal

- Conecta WiFi
- Optiene IP
- Mantiene la conexión y posee mecanismo de reconexión en caso de pérdida de red

MQTT TASK

Stack: 4096 Bytes
Prioridad: Low

- Crea Socket TCP
- Se suscribe a topic de control
- Publica el contenido de su cola **qMqttTx** en el topic correspondiente, que también se incluye en la cola
- KeepAlive cada 100 ms

UART CONFIG TASK

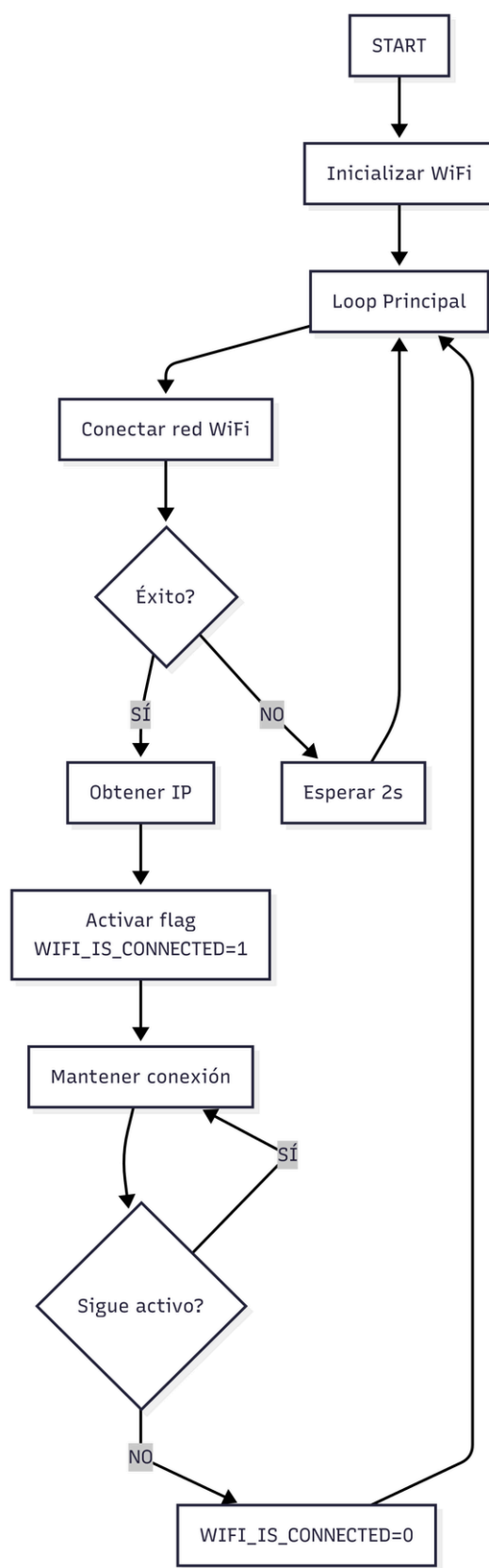
Stack: 5120 Bytes
Prioridad: Low

- Recepción char-by-char
- Usa función auxiliar para parsear comandos
- Encolado y mecanismo de Wake Up
- Soporte a topic de control MQTT y envíos por UART directamente

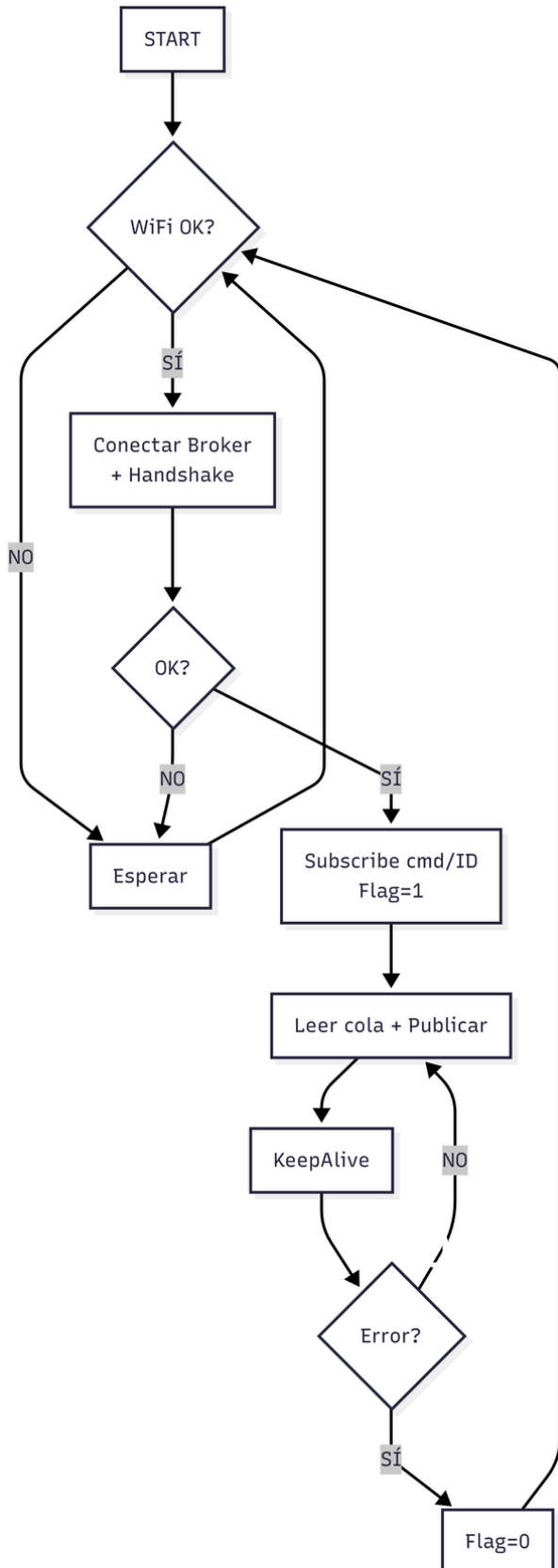
TAREAS COMUNES: WiFi, MQTT y Configuración

07

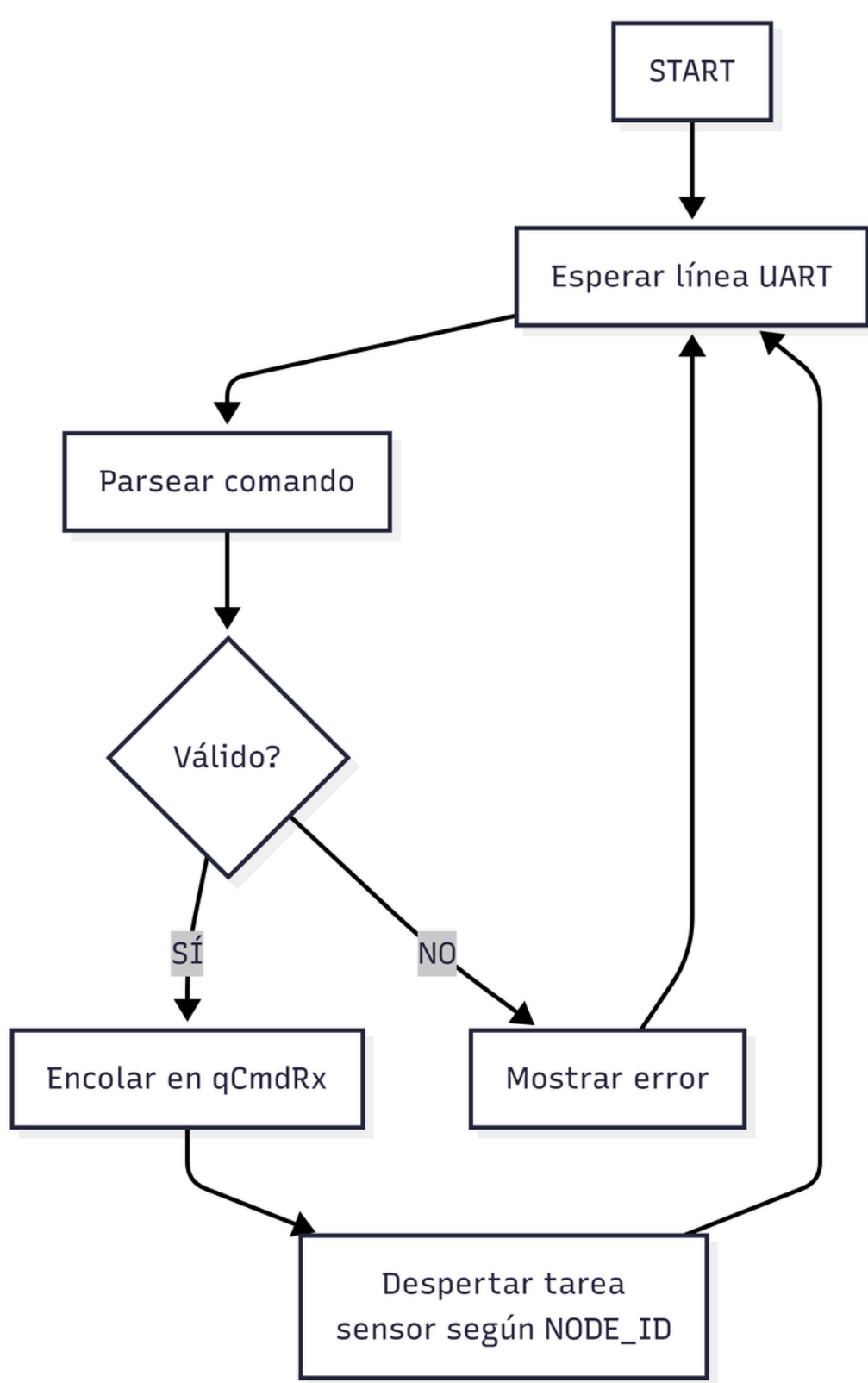
WIFI TASK



MQTT TASK



UART CONFIG TASK



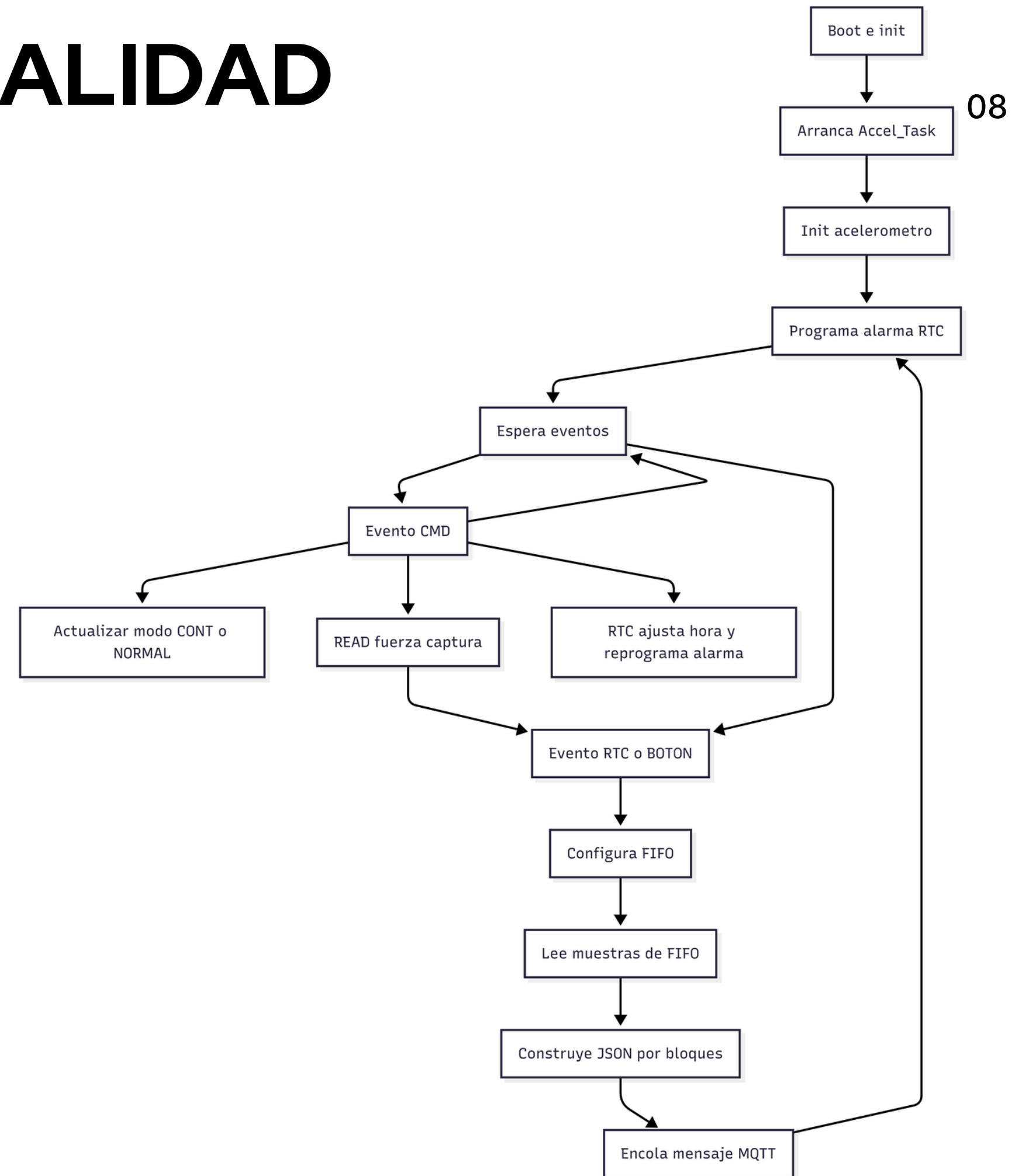
NODO 1 [ACC]: FUNCIONALIDAD

ACC TASK

Stack: 1024 Bytes

Prioridad: Normal

- Inicializa acelerómetro (BSP_ACCELERO_Init)
- Captura por FIFO con watermark (64 muestras o 1024)
- Despierta por flags: RTC / Botón / Comando (UART+MQTT)
- Modos: NORMAL (64 muestras) y CONT (1024 muestras)
- READ fuerza captura inmediata (como evento RTC)
- Publica JSON (t0, fs=52, seq, total, z[]) en bridge/accel/<id> vía cola MQTT



NODO 1 [ACC]: FIFO

09

LSM6DSL_FIFOCONFIG
(UINT16_T
SAMPLES_XYZ)

- Configura ODR=52Hz y FS=±2g en el acelerómetro
- Programa watermark = samples_xyz · 3 words (X,Y,Z)
- Habilita FIFO con modo FIFO + ODR FIFO
- Activa interrupción FIFO watermark en INT1 (evento de “bloque listo”)

LSM6DSL_FIFORESET
(VOID)

- “Vacía” la FIFO antes de capturar
- Cambia a BYPASS (000) para descartar datos antiguos
- Vuelve a FIFO mode (001) para empezar una captura limpia
- Evita mezclar muestras viejas con nuevas

LSM6DSL_FIFOGET
LEVELWORDS
(VOID)

- Lee cuántos words hay dentro de la FIFO
- Permite comprobar si ya hay datos (aunque se pierda la IRQ)
- Útil para el “fallback” por timeout

LSM6DSL_FIFOREADXYZ
RAW(INT16_T *X, INT16_T
*Y, INT16_T *Z)

- Lee 1 muestra XYZ directamente desde FIFO
- Accede a FIFO_DATA_OUT_L (6 bytes)
- Reconstruye X, Y, Z (2 bytes cada eje)
- Se repite hasta completar el bloque deseado

NODO 2 [ENV]: FUNCIONALIDAD

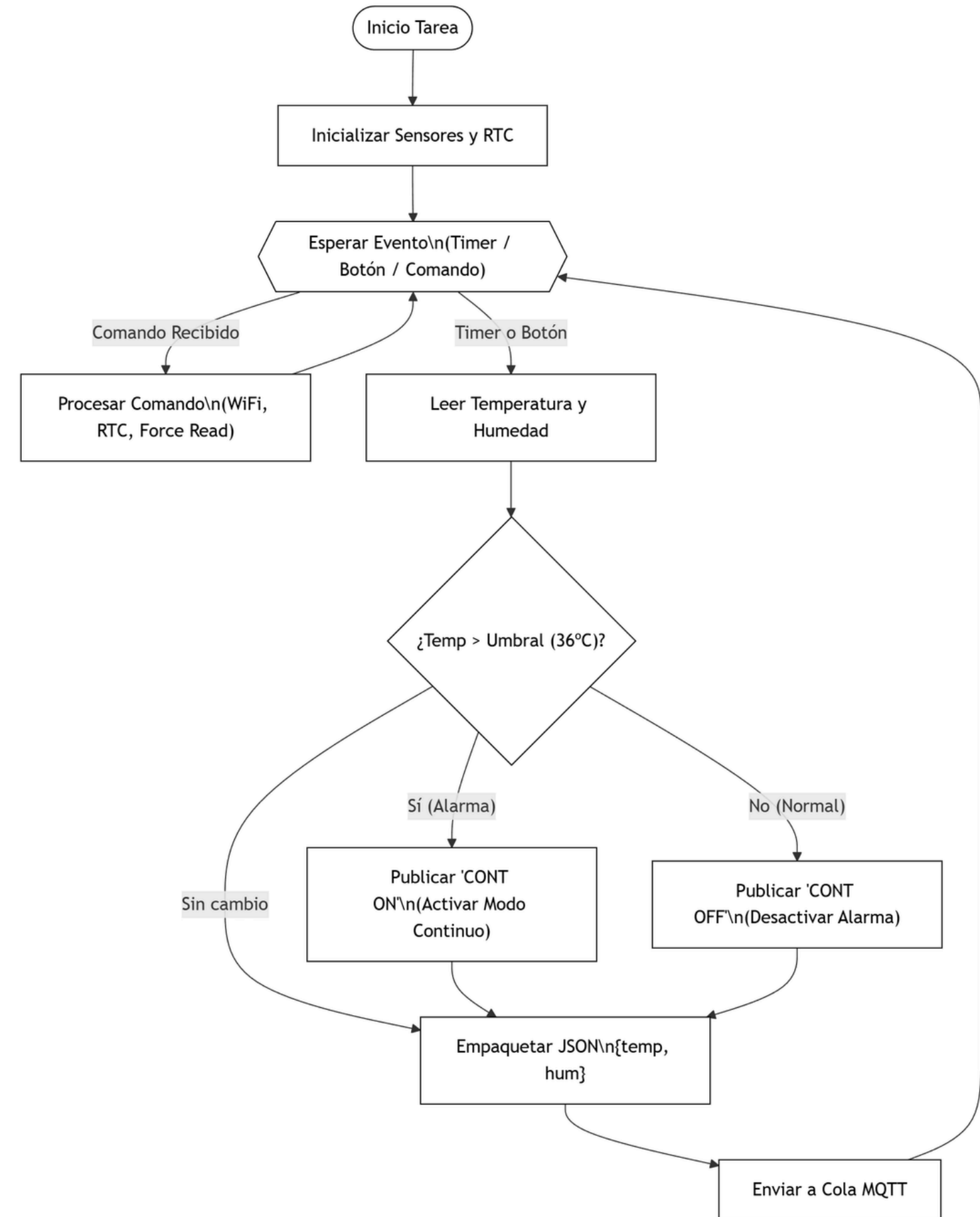
10

ENVIRONMENT READ TASK

Stack: 512 Bytes

Prioridad: Normal

- Inicializa sensor de temperatura y humedad (BSP_TSENSOR_INIT, BSP_HSENSOR_INIT)
- Despierta por flags: RTC / Botón / Comando (UART+MQTT)
- Comprueba si la temperatura es superior al umbral y envía alarma en caso necesario
- READ fuerza captura inmediata (como evento RTC)
- Publica JSON en bridge/env/<id> vía cola MQTT



NODO 2 [ENV]: FUNCIONES

11

PROGRAM_ALARM_RTC()

- Inicializa la RTC
- Calcula hora de la siguiente alarma.
- Programa la siguiente alarma
- Activa la interrupción de la alarma

BSP_TSENSOR_INIT()

- Inicializa el sensor de temperatura
- Comprueba si el sensor se ha inicializado correctamente

BSP_HSENSOR_INIT()

- Inicializa el sensor de humedad
- Comprueba si el sensor se ha inicializado correctamente

BSP_READ_TSENSOR_TEMP()

- Realiza una medida de temperatura
- Devuelve la temperatura como un float
- Posteriormente se procesa como un integer

BSP_READ_HSENSOR_HUMIDITY()

- Realiza una medida de humedad
- Devuelve la humedad como un float
- Posteriormente se procesa como un integer

```
{"id":1,"msg_id":2,"origen":"0","temp":213,"hum":74}
```

INTERFAZ DE CONTROL: UART Y MQTT

12

Canal de entrada:

- **UART terminal** (comandos escritos)
- **MQTT:** bridge/cmd/<id>

bridge/cmd/1 → Nodo 1 (ACC)

bridge/cmd/2 → Nodo 2 (ENV)

Acelerómetro (Nodo 1 – ACC)

- **CONT ON** → Activa **modo continuo** (captura larga y publicación por MQTT)
- **CONT OFF** → Vuelve a **modo normal** (captura corta)
- **READ** → **Forzar captura inmediata** (sin esperar RTC/botón)

WiFi (Nodo destino: ACC o ENV)

- **WIFI <ssid> <pass>** → Cambia credenciales y **reconecta**

Ejemplo: WIFI MiRed 12345678

RTC (Nodo destino: ACC o ENV)

- **RTC YYYY-MM-DD HH:MM:SS** → Ajusta fecha/hora (formato string)

Ejemplo: RTC 2026-02-08 19:30:00

- **RTC YYYY M D H M S** → Ajusta fecha/hora (por campos)

Ejemplo: RTC 2026 2 8 19 30 0

Comunicación automática ENV → ACC (alerta)

- **ENV publica en bridge/cmd/1:**

CONT ON cuando hay alerta

CONT OFF cuando termina la alerta

LOGS POR MQTT

EN DEMO

CONCLUSIONES

14

Sistema Modular y Escalable

IoT basado en FreeRTOS, tareas separadas (WiFi, MQTT, sensores, UART).

Comunicación MQTT Estable

Publicación de medidas, suscripción a comandos, acciones remotas.

Aprovechamiento del FIFO

Acelerómetro LSM6DSL reduce carga de CPU, capturas continuas.

Reacción a Eventos

Alarmas RTC, botón físico, interacciones entre nodos (ENV dispara ACC).

Observabilidad Mejorada

Sistema de logs (UART/SWV) y publicación por MQTT para depuración remota.

MEJORAS FUTURAS

15

Conexión Robusta

Recuperación automática de conexión ante caídas de WiFi o MQTT, evitando bloqueos y manteniendo la operatividad.

Confirmación de Comandos

Respuestas de "OK" tras cada comando para asegurar recepción y ejecución, mejorando la fiabilidad remota.

Estado Operativo Visible

Publicar el modo de operación actual (NORMAL/CONT), WiFi y MQTT en un tópico dedicado para monitorización transparente.

Gestión Eficiente de Datos

Optimizar el envío de datos: resúmenes en modo normal y datos completos solo con alertas, reduciendo el tráfico.

Seguridad Reforzada

Mejorar la seguridad con MQTT TLS o broker privado para proteger la integridad y confidencialidad de los datos.

Diagnóstico y Salud del Sistema

Integrar watchdog para estabilidad y funcionamiento continuo.

MUCHAS GRACIAS POR SU ATENCIÓN
¿PREGUNTAS?